Week 5 Progress Report — RESOLV-HQ

Agent Architecture and Bounded Autonomy

Date: 2 Oct 2026

Summary

The bounded agent loop itself (Sense → Plan → Act → Observe → Respond, MAX_ITERATIONS = 4, four approved tools, approval hand-off) was built and committed on 22 Sept, a week before this slot. This week formalised it: the Agent Task Contract and the extended architecture diagram are done. The execution-trace deliverable is only partly met: I captured one live-model success and one real failure/recovery, but not the three-trace set the brief wants, because both model providers became unavailable during capture. That is the headline risk going into Week 6.

Objectives vs. achievements

Objective (Week 5 brief)	Status	Evidence
One task that benefits from multi-step decision-making	Complete	Resolve-or-route a support query using lookups chosen by the model (agent-task-contract.md §1)
Sense → Plan → Act → Observe → Stop/Re-plan design	Complete	Contract 5; diagram extension in architecture-diagram.md
Max iterations, approved tools, stop conditions, hand-off conditions	Complete	Contract 2, 4, 6, 7 — MAX_ITERATIONS = 4; 4-tool closed set; 5 exit paths; agent_approvals for any draft
Implemented workflow	Complete	resolv-hq-backend/src/lib/react-agent.ts — runReActLoop() (direct orchestration, no framework)
Agent Architecture Diagram	Complete	Extended in place (not redrawn): five-phase subgraph, iteration cap, all exit paths, approval hand-off
Customer/staff behave identically	Complete (code-verified)	Staff caller adds one prompt sentence only; same tools, cap, boundary, scoping (Contract §8). Not yet confirmed with a live model
≥ 3 execution traces incl. one failure/recovery	Partial: 2 of 3	week-5-execution-traces.md: T1 live success (Gemini, grounded and cited, but the model called no tools); T2 real failure/recovery (both providers failed → keyword fallback). Missing: a live tool-calling trace and a boundary/iteration-cap trace

Deliverables

#	Deliverable	File	Status
1	Agent Task Contract	docs/agent-task-contract.md	Complete
2	Architecture diagram with agent loop	docs/architecture-diagram.md	Complete (extended)
3	Execution traces	evidence/traces/week-5-execution-traces.md	Partial
4	Week 5 progress report	docs/week-5-progress-report.md	This document

Key decisions

- **The contract is authoritative.** Rather than editing three older reports, the contract consolidates them and says it wins on any difference.
- **Diagram extended, not replaced.** Weeks 1–4 history stays intact for Week 6–7 additions.
- **Traces labelled by what they actually prove.** T1 is explicitly "no tools called", and T2 is explicitly gateway-level, not tool-level, recovery. I did not simulate the missing traces.

Risks and challenges

Risk	Detail	Mitigation / next step
No working live model (P0, now sharper)	The Anthropic key is registered (25 Sep) but returns 400 credit balance is too low; Gemini's free tier allows only 20 requests and returns 429 mid-run. Every live-evidence task (Week 5 traces, Week 3 15-case RAG eval, Week 7 30-scenario set) depends on this	Add Anthropic credit (or a paid Gemini tier) before Week 6 starts; then re-run the missing traces, ideally in one batch to stay within quota
Traces don't exercise tools	A live model answered a query from pre-loaded context with zero tool calls, so a trace can look "live" without testing Act/Observe	Design trace queries whose answer is not in the pre-loaded context (e.g. the caller's open-request status, or an escalation)
Iteration cap and boundary exits unproven live	Verified by code reading and offline smoke tests only	Capture them once a model is available; if an honest iteration-cap trace is hard to provoke, add a stubbed-provider unit test and label it as such
Test-data caveat	Traces used a small synthetic knowledge base and request, not production data	Stated in the trace file's method section

Plan for Week 6 (memory and MCP-style interface)

1. Restore a working provider, then capture the two missing traces (live tool-calling; boundary or iteration-cap) and a live customer-vs-staff comparison; update the trace file and contract 9.

2. Wire approved customer_memory_facts into the agent's context (the one remaining non-trivial code task), following the “informative, never authorizing” rule.
3. Add the MCP-style interface annotation to the Tools subgraph and turn the dashed Loaders → Memory edge solid.
4. Write the Memory Design and Data Handling Note once memory is wired.